Constraint Satisfaction Problems

CS3263

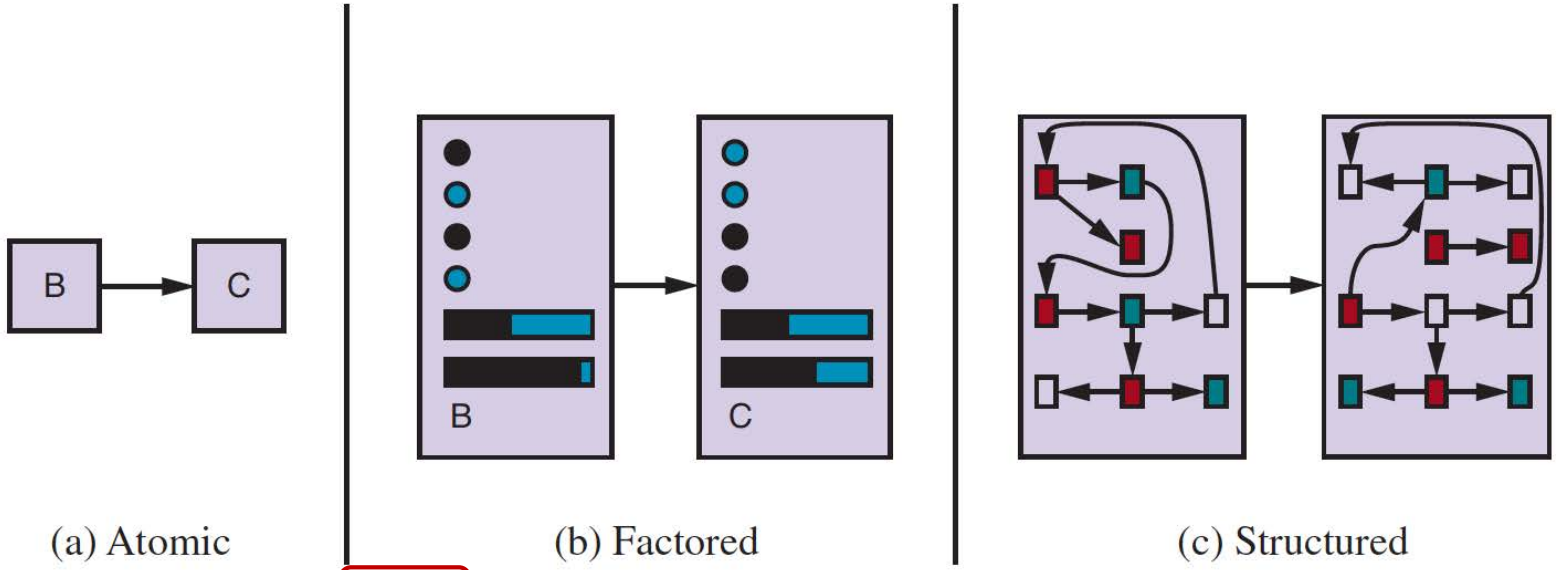
Foundations of Artificial Intelligence

Version 4.0

Topics

- Constraint Satisfaction Problems
- Constraint Propagation
- Backtracking Search in CSPs
- Improving Backtracking
  - Intelligent Backtracking
  - Topological Sorting and Arc Consistency in Trees
  - *Learning*
  - *Exploiting CSP Structures*
- Local Search in CSPs

Representing States and Transitions



(a) Atomic

Search, game-playing

(b) Factored

CSPs, planning, propositional logic, Bayesian networks, neural networks

(c) Structured

First order logic, databases, probabilistic programs

Source: RN Figure 2.16

Martian Adventures Continue

- Persy must collect samples from three Martian sites:
  - A, B, and C.
- Mission of planning a route that:
  - Starts and ends at the base station.
  - Visits each site exactly once.
  - Follows an ordering rule:
    - Site A must be visited before site C
  - Respects a battery constraint:
    - Total travel distance must not exceed 70 distance units.

Image source: ChatGPT4o

Constraint Satisfaction Problem (CSP)

Variables:

- Define three variables representing the visitation order:
  - Position1 (first site visited)
  - Position2 (second site visited)
  - Position3 (third site visited)
  - Domain for each variable: {A, B, C}.

Constraints:

1. All-Different Constraint:
   - Each site must appear exactly once in the route.
2. Ordering Constraint:
   - The index of A must be less than the index of C (i.e., A is visited before C).
3. Battery (Distance) Constraint:
   - Sum of the distances along the route (starting at base, visiting the sites in order, and returning to base) must be ≤ 70.

Distance Data:

- Assume the following distances (in arbitrary units):
  - From Base to Sites:
    - Base → A: 10
    - Base → B: 20
    - Base → C: 15
  - Between Sites (symmetric):
    - A ↔ B: 15
    - A ↔ C: 10
    - B ↔ C: 15
  - Return from Site to Base:
    - Use the same numbers as "Base → Site."
- What would be a solution to this CSP?
  - A possible assignment to the three variables

Solving the CSP

Let's test one possible assignment:

- Assignment:
  - Position1 = A
  - Position2 = B
  - Position3 = C
- Step 1: Check the Ordering Constraint

  Since A is assigned to Position1 and C to Position3 the requirement "A before C" is satisfied.
- Step 2: Check the All-Different Constraint

  All positions are filled with distinct sites (A, B, C).
- Step 3: Verify the Battery (Distance) Constraint

  Calculate the total travel distance:
  - Base → A: 10
  - A → B: 15
  - B → C: 15
  - C → Base: 15
  - Total Distance = 10 + 15 + 15 + 15 = 55 units
  - Since 55 ≤ 70, the battery constraint is met.

Conclusion

- Assignment [A, B, C] is a valid solution to the CSP for Persy's sampling mission:

  It meets all the constraints:

  - Each site is visited exactly once.
  - Site A is visited before C.
  - The total route distance (55 units) is within the 70-unit battery limit.

- A CSP can be

  set up by:

  - defining variables,
  - specifying domains and constraints

  and then solved by:

  - checking candidate assignments (often via Constraint Propagation techniques)

  to find a route that satisfies all mission constraints or requirements

Real-World CSPs

- Automated planning
- Automated scheduling
- Product configuration
- Resource allocation
- Lexical disambiguation – identifying meaning of a word..
- Musicology – automated generation of musical structures …
- …

Source: Wikipedia

Constraint Satisfaction Problems

Problem formulation and solution

Constraint Satisfaction Problems

- Each CSP consists of three components: X, D, and C:
  - Variables $X = \{X_1, \dots, X_n\}$
  - Domains $D = \{D_1, \dots, D_n\}$, one for each variable
    - Domain D_i : a set of allowable values, $\{v_1, \dots, v_n\}$, for variable X_i
  - Constraints $C = \{C_1, \dots, C_n\}$, a set of constraints over X and D
    - Constraint C_i: $\langle scope,\ rel \rangle$: defined over variables X with domains D

$scope$: tuple of variables in constraint

rel : relation defines value that those variables can take on

- Example:
  - X_1 and X_2 both have domain $\{1, 2, 3\}$; constraint $X_1 > X_2$:
  - $\langle (X_1, X_2), \{(3,1), (3,2), (2,1)\} \rangle$
  - $\langle (X_1, X_2),\ X_1 > X_2 \rangle$

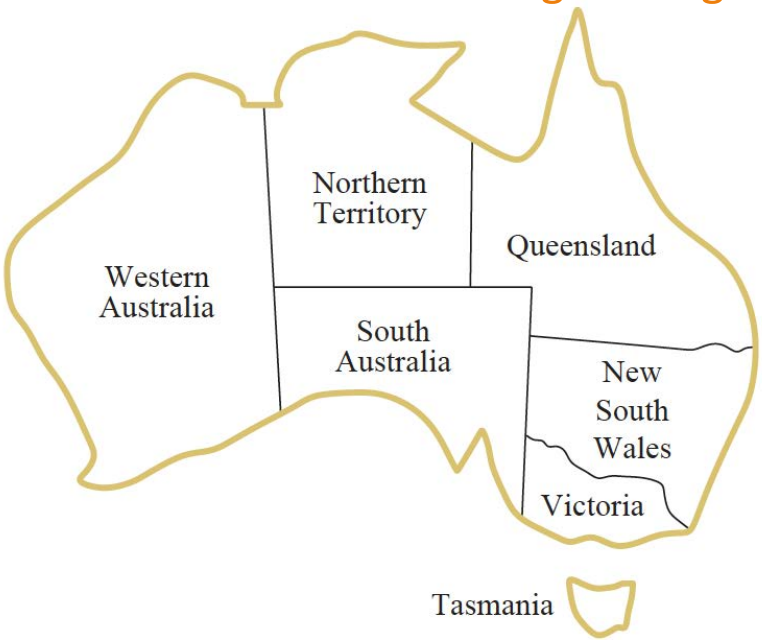
Relation as explicit set of all tuples of values that satisfy constraint, or function that computes whether tuple is member of relation.

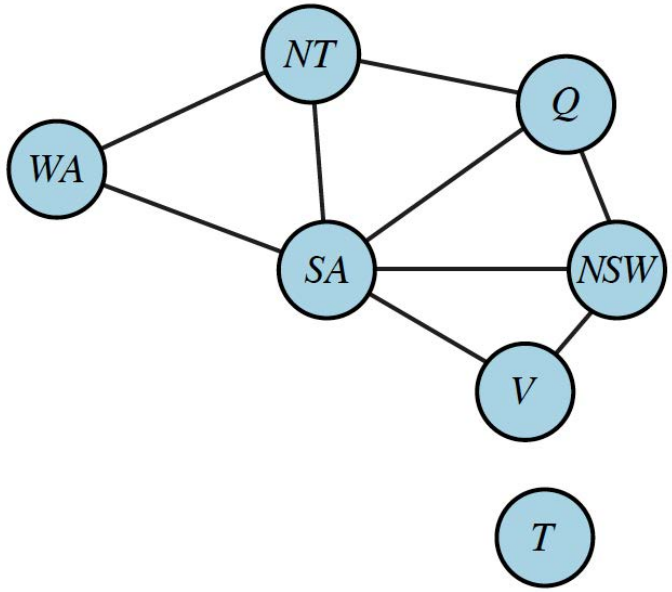
Solving CSPs

- Goal state: a consistent, complete assignment
  - Each variable has value $\{X_i = v_i, X_j = v_j, \ldots\}$ that does not violate any constraints
- CSP solvers:
  - Exploit structure of states
  - Use general heuristics for solution search
  - Identify constraint-violating variable/value combinations to reduce search space
  - Deduce actions and transition models from problem descriptions
  - NP-complete in general; efficient algorithms exist for subclasses

Example: Map Coloring (for Australia)

How to assign colors to each region so that
no neighboring regions have the same color?

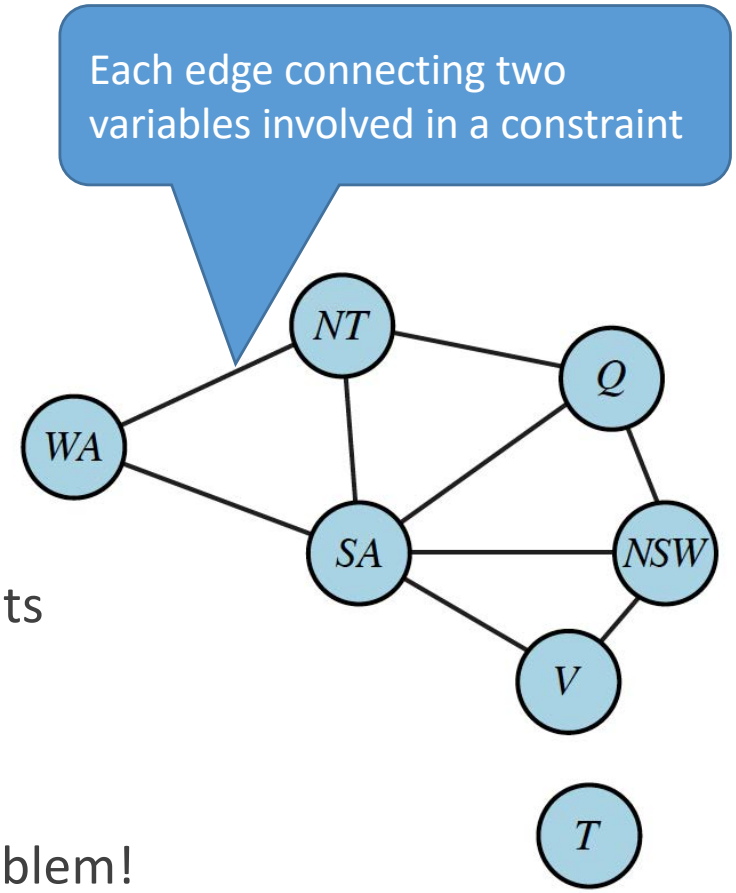




Source: RN Figure 6.1

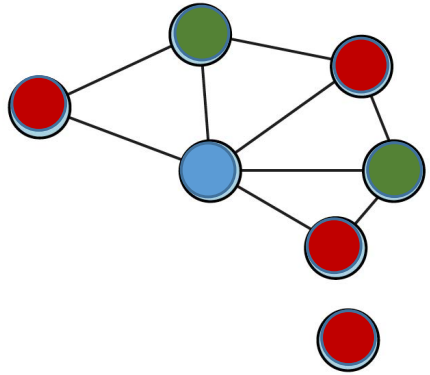
Constraint Graphs

- Binary CSP:
  - Each constraint relates (at most) two variables – unary and binary constraints
- Binary constraint graph:
  - Nodes are variables, edges are constraints
- CSP search algorithms:
  - Use graph structure to speed up search
  - E.g., Tasmania is an independent subproblem!



Source: RN Figure 6.1b

Example: Map Coloring



- Variables:
  - $X = \{WA, NT, Q, NSW, V, SA, T\}$
- Domains:
  - $D_i = \{red, green, blue\}$
- Constraints: adjacent regions must have different colors
  - $C = \{SA \neq WA, SA \neq NT, \ldots\}$ or $\{\langle(SA, WA), SA \neq WA\rangle, \ldots\}$
  - Implicit: $SA \neq WA, \ldots$
  - Explicit: $\{(red, green), (red, blue), \ldots\}$

9 constraints

- Solutions – assignments satisfying all constraints, e.g.,
  $\{WA = red, NT = green, Q = red, NSW = green, V = red\ SA = blue, T = red\}$.

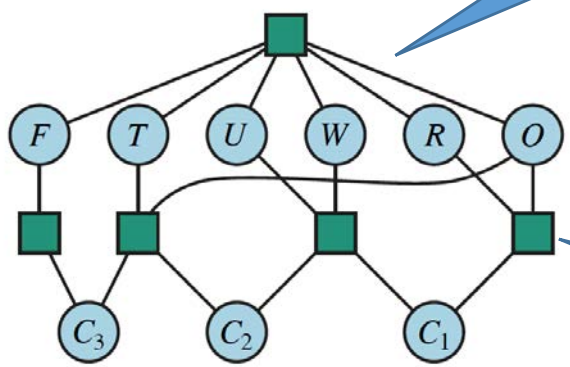
Example: Cryptarithmetic

**Any constraint can be reduced to a set of binary constraints if enough auxiliary variables are introduced or use "dual graph"

- Puzzle:
  - Each letter (variable) represents a distinct digit
  - Find substitutions for arithmetically correct results
- Variables:
  - $F, T, U, W, R, O, C_1, C_2, C_3$
- Domains:
  - $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ for variables
  - {0, 1} for the C's
- Constraints:
  - Global constraint: $Alldiff(F, T, U, W, R, O)$
  - $O + O = R + 10.C_1$
  - $C_1 + W + W = U + 10.C_2$
  - $C_2 + T + T = O + 10.C_3$
  - $C_3 = F$

$$\begin{array}{r} T\ W\ O \\ +\ T\ W\ O \\ \hline F\ O\ U\ R \end{array}$$

Constraint Hypergraph



An n-ary constraint

Source: RN Figure 6.2

Absolute Constraints

- Absolution constraint – violation rules out a potential solution
- Unary constraint
  - E.g., $\langle (SA), SA \neq green \rangle$
- Binary constraint
  - E.g., $SA \neq NSW$
- Higher order constraint
  - E.g., Ternary constraint $Between(X, Y, Z)$: $\langle (X, Y, Z), X < Y < Z \rangle$
- Global constraint
  - Involving an arbitrary number of variables
  - E.g., $Alldiff$ – all variables involved in the constraint must have different values

A binary CSP
- CSP with only unary and binary constraints
- Can be represented as a constraint graph

Preference Constraints

- Preference constraint – indicates which solutions are preferred
- Example: Class scheduling
  - Absolute constraint: no professor can teach two classes at the same time
  - Preference constraint: Prof. Nut prefers teaching in the morning (Allowable solution: Prof. Nut teaches at 6:30pm)
- Constrained optimization problem (COP)
  - Preference constraints encoded as costs on individual variable assignments
  - Solving CSP as optimization search
  - E.g., linear programs

Higher Order Constraints

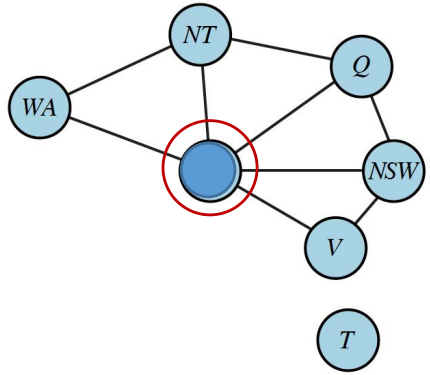
- Global constraint
  - Involves multiple variables (not necessarily all).
  - $Alldiff$
- Resource constraint
  - $Atmost$
- Bounds propagation
  - Arithmetic constraints on $D_1, D_2, \ldots, D_n$
  - A CSP is bounds-consistent if:

    For every variable X, both its lower and upper bounds have a valid matching value in another variable Y.

Normal Search vs CSP

- Normal Search Problems
  - Goal: A single state that is either valid or invalid.
  - Search algorithms do not use the internal structure of states.
  - Examples: Pathfinding, puzzles.
- Constraint Satisfaction Problems (CSPs)
  - Goal: A complete assignment of values to variables where all constraints are satisfied.
  - Uses constraints to reduce the search space.
  - Solutions can be satisfiable, partially satisfiable, or unsatisfiable.
  - Examples: Sudoku, scheduling, map coloring.
- Key Difference
  - Normal Search: Explores full states blindly.
  - CSPs: Uses constraints to guide and optimize search.

Why use CSPs?



- Efficient Search via Constraint Propagation
  - Constraint Propagation: Eliminates invalid choices early.
  - Efficient Search: Focuses only on relevant variables.
- Example:
  - Choose $\{SA = blue\}$ - all five neighboring variables *cannot* be blue
  - Direct search: assignments
  - CSP pruning: assignments
- Key Benefit:
  - CSP methods drastically reduce the search space, making problem-solving more efficient.

Constraint Propagation

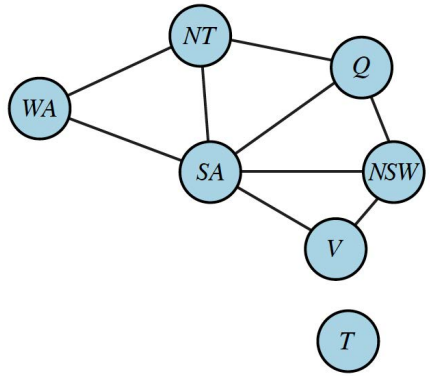
Inference in CSPs

Solution Search in CSP

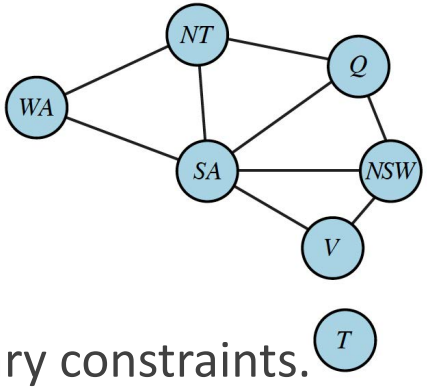
- Successors in Search Tree Generation
  - Choose a new variable assignment or apply constraint propagation
- Constraint Propagation
  - Reduces legal values for variables using constraints
  - Shrinks the search space and limits choices
  - Can be a preprocessing step or work alongside search
  - Sometimes solves the entire problem without searching
  - Core idea: maintain local consistency

Constraint Graph and Local Consistency

- Constraint Graph:
  - Nodes = Variables
  - Edges/Arcs = Binary constraints
- Local Consistency:
  - Maintains consistency in parts of the graph
  - Removes inconsistent values
- Types of Local Consistency:
  - 1-consistency → Node consistency
  - 2-consistency → Arc/Edge consistency
  - 3-consistency → Path consistency
  - K-consistency → Generalized form



Node Consistency



- Node Consistency (1-Consistency)
  - A variable is node-consistent if all its values satisfy the unary constraints.
  - A CSP is node-consistent if all variables are node-consistent.
- Example: Map Coloring Problem
  - Scenario: South Australians dislike $green \rightarrow SA \neq green$
  - Initial domain: $SA = \{red, green, blue\}$
  - Node consistency: Remove $green$, leaving $SA = \{red, blue\}$
- Preprocessing
  - Remove unary constraints by reducing variable domains.

Arc Consistency

- Arc consistency (2-consistency)
  - A variable X_i is arc-consistent with respect to variable X_j if:
    - For every value x in D_i, there exists at least one value y in D_j that satisfies the binary constraint $C(X_i, X_j)$.
  - A CSP is arc-consistent if all variables are arc-consistent with all the variables they share a binary constraint with..

Arc Consistency

- Example:
  - Consider constraint $Y = X^2$,
    where domain of X, Y is the set of decimal digits: {0, 1, 2, …, 9}
  - Explicit constraint: $\langle (X, Y), \{(0,0), (1,1), (2,4), (3,9)\} \rangle$.
  - To make X arc-consistent with respect to Y:
    Reduce X's domain to $\{0,1,2,3\}$.
  - To make Y arc-consistent with respect to X:
    Reduce Y's domain to $\{0,1,4,9\}$.
  - Whole CSP is now arc-consistent.

Arc Consistency Algorithm AC-3

Arc-consistent CSP equivalent to original CSP: faster to search; variables have smaller domains

```
function AC-3( csp) returns false if an inconsistency is found and true otherwise
  queue ← a queue of arcs, initially all the arcs in csp

  while queue is not empty do
    (X_i, X_j) ← POP(queue)
    if REVISE(csp, X_i, X_j) then
      if size of D_i = 0 then return false
      for each X_k in X_i.NEIGHBORS - {X_j} do
        add (X_k, X_i) to queue
  return true

function REVISE( csp, X_i, X_j) returns true iff we revise the domain of X_i
  revised ← false
  for each x in D_i do
    if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j then
      delete x from D_i
      revised ← true
  return revised
```

Initial queue contains all arcs

Make X_i arc-consistent wrt X_j

Move to next arc if D_i unchanged

If D_i is revised, add to queue all arcs of neighbors of X_i for further reduction

Keep checking and reducing until queue is empty

If D_i is reduced to nothing, Result is CSP has no consistent solution

Result is arc-consistent CSP or solution

Source RN Figure 6.3

Example: Sudoku

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

- Variables:
  - 81 variables (squares)
- Domains:
  - Empty square: $\{1,2,\ldots,9\}$
  - Prefilled square with v: $\{v \in \{1,2,\ldots,9\}\}$
- Constraints: (on unit)
  - 9-way $Alldiff$ for each row
  - 9-way $Alldiff$ for each column
  - 9-way $Alldiff$ for each box
- Can AC-3 solve the puzzle?

Backtracking Search for CSPs

Search for solutions over partial assignments

Backtracking Search in CSPs

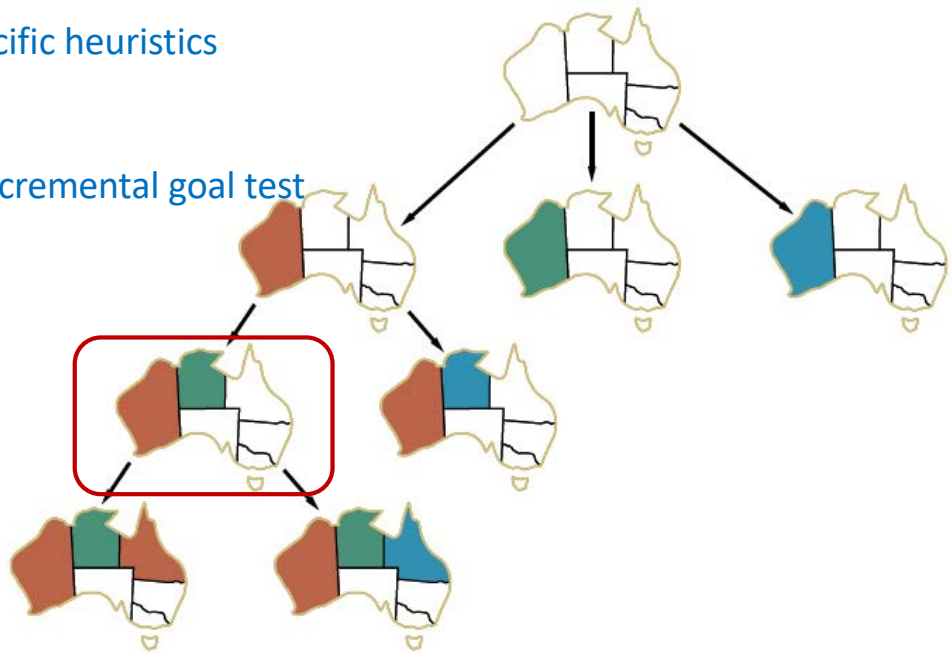
- Motivation
  - Constraint propagation may end with some variables with multiple possible values.
- Backtracking Search in CSPs
  - Works with partial assignments to explore valid solutions
  - Depth-first search (DFS) variant designed for CSPs.

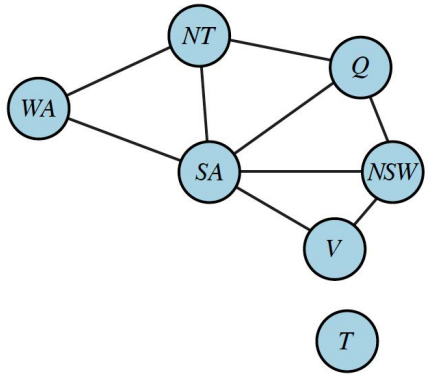
Vs. Local search:
Works over complete assignments

- Key Concepts:
  - States: Partial assignments (variables assigned so far).
  - Actions: Assign a value to an unassigned variable.
  - Initial state: Empty assignment {}.
  - Goal test: A complete assignment satisfying all constraints.
  - Search tree: Leaf nodes at depth n represent complete assignments for n variables.
  - Efficient pruning techniques (e.g., forward checking, arc consistency) can improve performance.

Example: Backtracking Search Tree

- Altering factored representation of state (assignment)
- General and domain specific heuristics
- One variable at a time
- Commutativity
- Constraint checking as incremental goal test





Source: RN Figure 6.6

Backtracking Search

```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
  return BACKTRACK(csp, { })

function BACKTRACK(csp, assignment) returns a solution or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences ← INFERENCE(csp, var, assignment)
      if inferences ≠ failure then
        add inferences to csp
        result ← BACKTRACK(csp, assignment)
        if result ≠ failure then return result
        remove inferences from csp
      remove {var = value} from assignment
  return failure
```

Repeatedly choose an unassigned variable

Try all values in variable domain to extend solution recursively

If succeeds, return solution

If fails, restore to previous assignment and try next value

Where to backtrack to?

Source: RN Figure 6.5

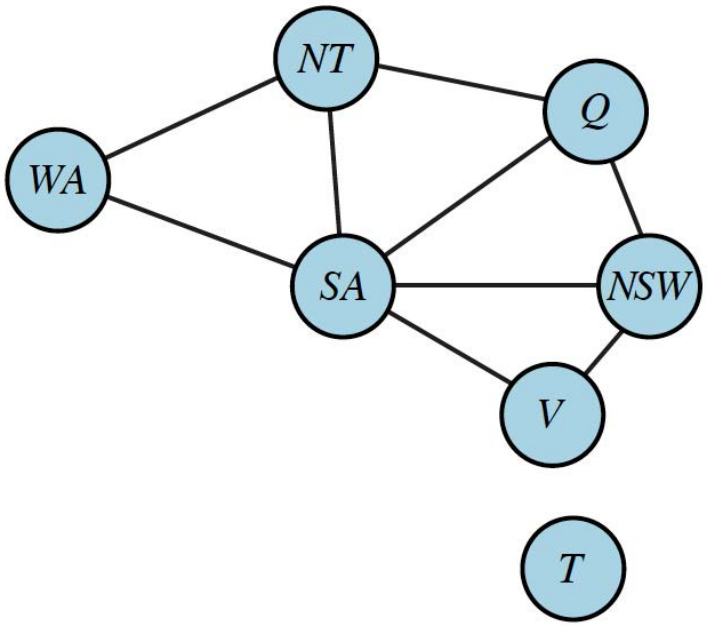
Improving Backtracking Search

Ordering, filtering, Intelligent backtracking

Ordering

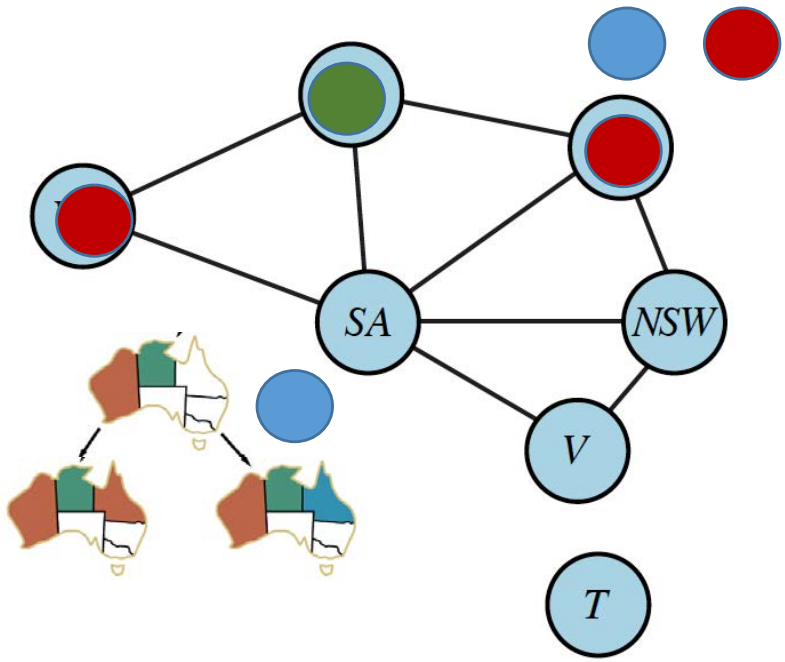
Which variable should be assigned next?

- Minimum remaining value heuristic (MRV):
  - Initially every region has 3 legal colors
  - *Not helpful in choosing first region to color in map*
- Degree heuristic:
  - $d(SA) = 5$
  - $d(T) = 0$
  - All other variables have degree 2 or 3



What order should variable values be tried?

- Least constraining value heuristic (LCV):
  - $\{WA = red, NT = green\}$
  - Next choice: Q
  - Not $blue$!
    - Eliminate last legal value for SA
    - (Q's neighbor)
  - $red \succ blue$ for Q



Filtering

Interleaving Search and Inference

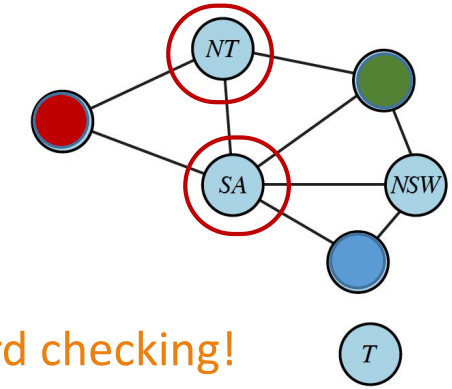
Before Search:

- Use AC-3 or other pruning techniques to reduce domains.

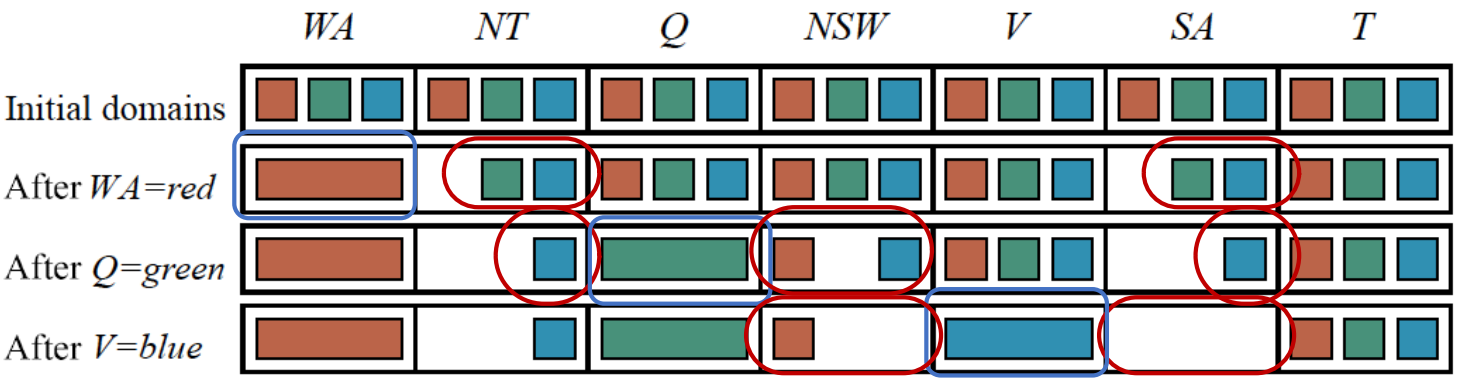
During Search:

- Inference:
  - Assigning a variable can reduce domains of neighboring variables.
- Forward Checking:
  - When X is assigned, ensure arc-consistency for each connected unassigned variable Y.
  - Remove values from Y's domain that conflict with X.
- Constraint Propagation:
  - Extends reductions to further variables but does not catch all inconsistencies.

Example: Forward Checking



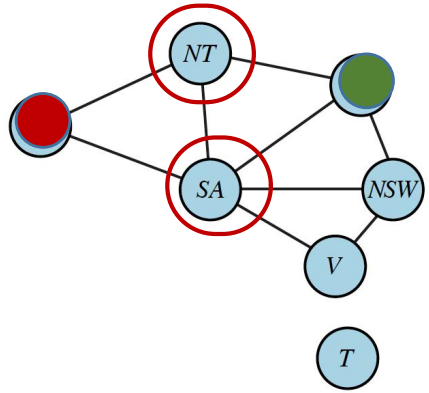
Potential improvement: Combine MRV heuristic with forward checking!



- Problem with Forward Checking: Consider the $Q = green$ row.
- WA and Q are arc-consistent, but both NT and SA are left with $blue$ as their only possible value
- $\{WA = red, Q = green, V = blue\}$ is inconsistent – backtrack!

Source: RN Figure 6.7

Example: Maintaining Arc Consist



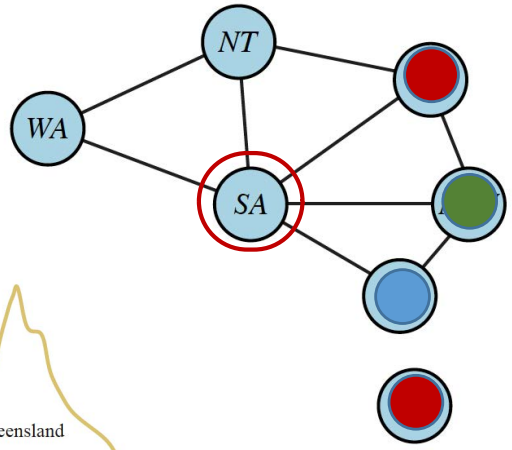
- MAC approach:
  - Instead of just pruning immediate neighbors like FC, MAC applies full arc consistency (AC-3) after each domain change.
- Solving map-coloring problem:
  - After assigning $WA = Red$, prune NT and SA as in FC.
  - After assigning $Q = Green$, NT and SA both get reduced to $\{Blue\}$.
  - Before assigning $V = Blue$, MAC enforces arc consistency:
    - It sees that NT and SA are both forced to $Blue$, but $SA \neq NT$.
    - → Conflict detected immediately! Backtrack before assigning V.

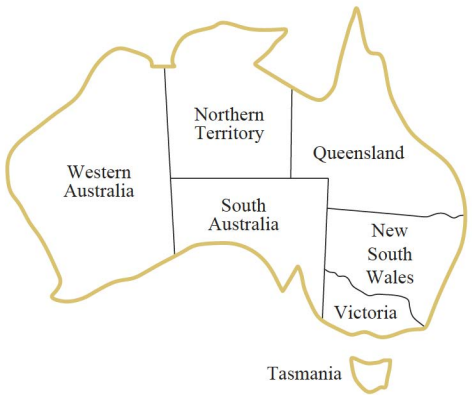
Maintaining Arc Consistency Algorithm

- Maintaining Arc Consistency (MAC) Algorithm
  - After assigning a variable X_i, AC-3 is applied to enforce arc consistency.
  - Starts with arcs (X_j, X_i) for all unassigned neighbors X_j.
  - Constraint propagation follows as in AC-3.
  - If any domain becomes empty, backtrack immediately.
- MAC vs. Forward Checking (FC)
  - MAC is more powerful: It recursively propagates constraints when domains change.
  - FC only checks immediate constraints, without further propagation.
  - MAC detects conflicts earlier, prunes the search space more efficiently

Intelligent Backtracking

Example: Chronological Backtracking





- Idea:
  - Backtracking in in map with fixed variable ordering:
    $$Q, NSW, V, \boxed{T}, SA, WA, NT.$$
  - Given partial assignment
    $$\{Q = red, NSW = green, V = blue, T = red\}.$$
  - Next variable to try: SA. What happens?
  - Chronologically backtrack to T and try a new color for Tasmania!
  - Does this work?
    - Recoloring Tasmania cannot possibly help in resolving the problem with South Australia!

Source: RN Figure 6.1

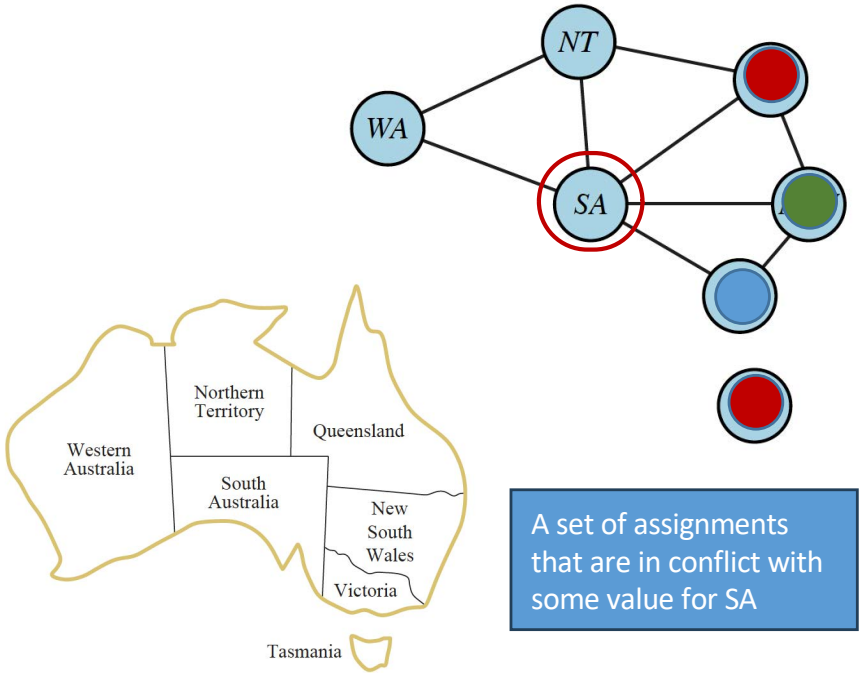
Intelligent Backtracking: Looking Backward

- Chronological Backtracking
  - On failure, backtrack to the most recent decision and try a different value.
- Conflict Set Backjumping
  - Conflict set of X_j : Variables whose assignments conflict with some values of X_j .
  - Backtrack to the most recent conflicting assignment instead of the last decision.

Forward checking derives conflict set directly

- Backjumping skips irrelevant choices, reducing unnecessary exploration.

Example: Conflict Set Backjumping



A set of assignments that are in conflict with some value for SA

- Idea:
  - Backtracking in in map with fixed variable ordering:
    $$Q, NSW, \boxed{V}, T, SA, WA, NT.$$
  - Given partial assignment
    $$\{Q = red, NSW = green, V = blue, T = red\}.$$
  - Next variable to try: SA. What happens?
  - Backtrack to a variable that conflicts with (makes impossible) a value of SA
  - Conflict set for SA:
    $$\{Q = red, NSW = green, V = blue\}$$
  - Backjumping backtrack to most recent assignment in conflict set
    - Jump over T and try a new value for V

Source: RN Figure 6.1

Local Search for CSPs

Search for solutions over complete assignments

Local Search for CSPs

- Main Idea
  - States: Complete assignments (every variable has a value).
  - Actions: Modify one variable's value at a time.
  - Min-Conflicts Heuristic: Choose the value with the fewest conflicts.
  - Goal Test: Assignment satisfies all constraints.
- Local Search Methods:
  - Can solve CSPs and extend to Constrained Optimization Problems (COPs).
  - Techniques like *hill climbing* and *simulated annealing* optimize the objective function.

Min-Conflicts Local Search

Iteratively resolves CSP violations by reassigning conflicted variables to minimize conflicts.

```
function MIN-CONFLICTS(csp, max_steps) returns a solution or failure
  inputs: csp, a constraint satisfaction problem
          max_steps, the number of steps allowed before giving up

  current ← an initial complete assignment for csp
  for i = 1 to max_steps do
     if current is a solution for csp then return current
     var ← a randomly chosen conflicted variable from csp.VARIABLES
     value ← the value v for var that minimizes CONFLICTS(csp, var, v, current)
     set var = value in current
  return failure
```

chosen at random or greedy assignment of minimally conflicted value

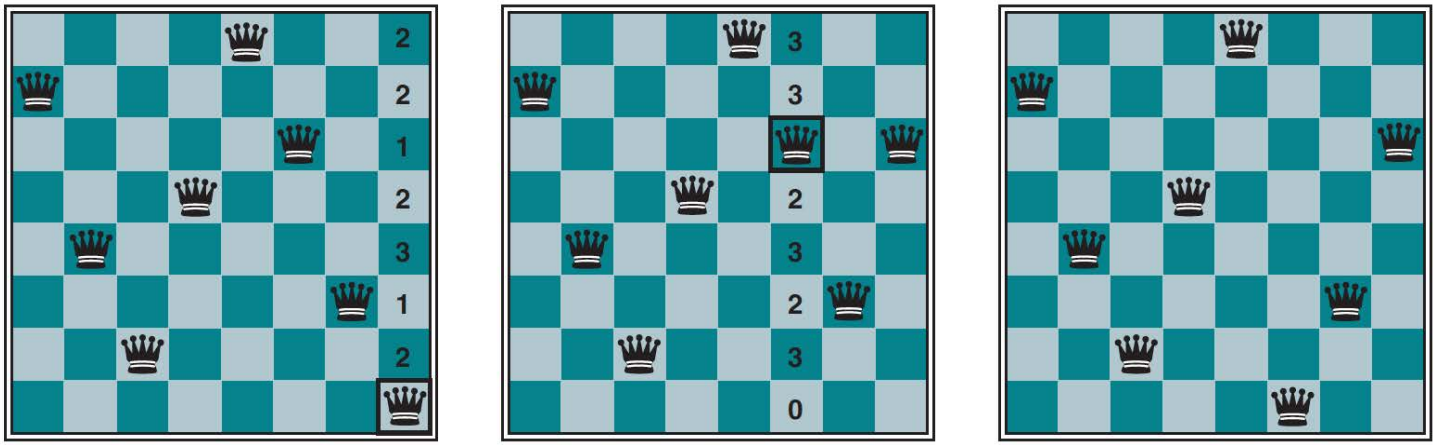
Counts no. of constraints violated by value v in current assignment

Source: RN Figure 6.9

Example: N-Queens Problem

States: 8 queens in 8 columns (8^8 = 16.8m)
Actions: move queen in column
Goal test: no attacks
Evaluation: c(n) = number of attacks

No. of conflicts shown. Move to the min-conflicts square, breaking ties randomly



N-queens is easy for local search because solutions are densely distributed throughout state space.

Source: RN Figure 6.8

Example: Airline Scheduling

- Challenges:
  - Thousands of flights, personnel assignments.
- Disruptions:
  - Weather can make schedules infeasible.
- Goal:
  - Repair schedule with minimal changes.
- Approach:
  - Local search starts from the current schedule.
- Why not backtracking?
  - Slower and requires more drastic changes to the schedule.

Exploiting CSP Structure

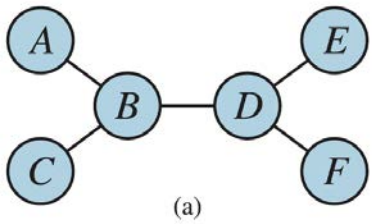
Topological sorting

Tree Structured CSPs

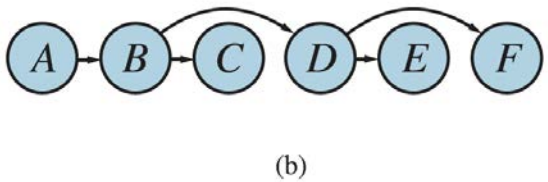
- Tree-Structured Constraint Graph
  - A tree has only one path between any two variables.
  - Tree-structured CSPs can be solved in linear time.
- Ensuring Arc Consistency
  - A CSP is directionally arc-consistent under an ordering of variables $X_1, X_2, \ldots, X_n$ if and only if every X_i is arc consistent with all X_j for $j > i$.
- Ways to Derive Tree Structures
  - Topological sorting (imposes a parent-to-child order in the tree).
  - Cutset conditioning (removes nodes to break cycles).
  - Tree decomposition (groups nodes to form a tree structure).

Topological Sorting to Solve Tree-Structured CSP

- Topological sorting:
  - Choose a root and order variables so parents precede children.
- Arc-consistency (Bottom-up):
  - From leaves to root, enforce consistency:
  - For $j = n$ to 2, Make–arc–consistent$(Parent(X_i), X_i))$
- Assignment (Top-Down):
  - Assign values consistently from root to leaves.
  - For $i = 1$ to n, assign X_i based on $Parent(X_i)$
- Time complexity
  - A tree with n nodes has $n - 1$ edges.
  - Arc consistency requires $O(n)$ steps, each checking up to d values for two variables.
  - Total time: $O(nd^2)$.



Tree-structured constraint graph



Topological sort

Source: RN Figure 6.10

Tree-CSP Solver Algorithm

Solves tree-structured CSPs in linear time using (bottom-up) arc consistency and (top-down) value assignment.

```
function TREE-CSP-SOLVER(csp) returns a solution, or failure
  inputs: csp, a CSP with components X, D, C

  n ← number of variables in X
  assignment ← an empty assignment
  root ← any variable in X
  X ← TOPOLOGICALSORT(X, root)
  for j = n down to 2 do
    MAKE-ARC-CONSISTENT(PARENT(X_j), X_j)
    if it cannot be made consistent then return failure
  for i = 1 to n do
    assignment[X_i] ← any consistent value from D_i
    if there is no consistent value then return failure
  return assignment
```

Top down topological sort

Bottom-up arc consistency checking

Source: RN Figure 6.11

Summary: CSPs

- CSP Formalism: A Specialized Search Problem
  - Defined by constraints, using general solving techniques.
  - States: Partial or complete assignments; Goal: Satisfy all constraints.
- Solution Approach:
  - Constraint propagation + backtracking or local search + exploiting structure
- Binary Constraints:
  - Any constraint can be reduced to binary.
  - Efficient solvers exist for binary CSPs.
- Widely used in real-world applications!

Homework

- Readings
  - RN Chapter 6, 6.1-6.33, 6.4, *6.5 (main introduction only)*

- (RN) Russell, S. and P. Norvig, Artificial intelligence: A modern approach. 4th ed. 2021: Pearson. [Kindle Edition] (Alternate: 3rd ed.)
  [Table of contents for 4th ed.: http://aima.cs.berkeley.edu/contents.html]